



UNIVERSIDAD DEL CAUCA

PROGRAMA DE INGENIERÍA ELECTRÓNICA Y TELECOMUNICACIONES

Diseño de Circuitos Digitales con VHDL

Segundo microproyecto

DOCENTE: Cristian Heidelberg Valencia

Integrantes: Camilo Andrés Luna

Fecha: 4 de Mayo de 2026

1 Introducción

El presente informe documenta el diseño e implementación de un sistema de memoria que integra una **ROM síncrona**, una **RAM síncrona** y un **decodificador BCD a 7 segmentos**, todo controlado por una **máquina de estados finita (FSM)** tipo Moore desplegada sobre la tarjeta de desarrollo **FPGA Altera DE0**.

El proyecto está estructurado en siete archivos VHDL que conforman una jerarquía modular:

1. `mem_pkg.vhd` — Paquete de tipos, constantes y subprogramas.
2. `rom_sync.vhd` — Memoria ROM síncrona de 16×8 bits.
3. `ram_sincrona.vhd` — Memoria RAM síncrona de 16×8 bits.
4. `divisor_frecuencia.vhd` — Divisor de frecuencia para 50 MHz.
5. `dec_7seg.vhd` — Decodificador BCD/Hex a display de 7 segmentos.
6. `memory_controller.vhd` — Controlador FSM tipo Moore (9 estados).
7. `memory_system_top.vhd` — Top-level estructural que conecta todos los módulos.

Adicionalmente se incluye el archivo `tb_memory_system.vhd` que implementa el *testbench* de verificación del sistema completo.

2 Descripción del sistema

2.1 Arquitectura general

El sistema sigue una arquitectura estructural donde la FSM actúa como controlador central. El top-level `memory_system_top` instancia todos los componentes y gestiona las señales de interconexiones. La figura 1 ilustra los bloques funcionales.

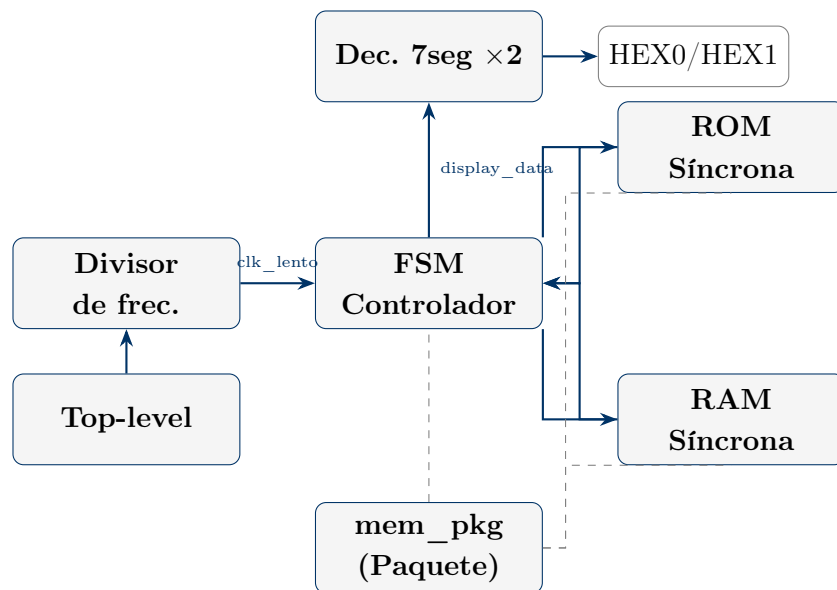


Figura 1: Arquitectura general del sistema ROM-RAM-FSM.

2.2 Parámetros globales del sistema

Cuadro 1: Constantes definidas en `mem_pkg.vhd`

Constante	Valor	Descripción
DATA_WIDTH	8	Ancho de palabra de datos (bits)
ADDR_WIDTH	4	Ancho del bus de direcciones (bits)
MEM_DEPTH	16	Número de posiciones de memoria (2^4)

3 Módulos VHDL

3.1 Paquete `mem_pkg.vhd`

El paquete centraliza definiciones reutilizadas por todos los módulos. Declara los subtipos `data_word` (8 bits) y `addr_word` (4 bits), el tipo `mem_t` (arreglo de 16 palabras), la función `addr_to_integer()` y el procedimiento `clear_control_signals()`.

```

1  package mem_pkg is
2      constant DATA_WIDTH : integer := 8;
3      constant ADDR_WIDTH  : integer := 4;
4      constant MEM_DEPTH   : integer := 2**ADDR_WIDTH;  -- 16
5
6      subtype data_word is std_logic_vector(DATA_WIDTH-1 downto 0);
7      subtype addr_word is std_logic_vector(ADDR_WIDTH-1 downto 0);
8      type mem_t is array (0 to MEM_DEPTH-1) of data_word;
9
10     function addr_to_integer(addr : addr_word) return integer;
11     procedure clear_control_signals(
12         signal rom_re : out std_logic;
13         signal ram_we : out std_logic;
14         signal ram_re : out std_logic
15     );
16 end package mem_pkg;
```

Listing 1: Fragmento clave de `mem_pkg.vhd`

Función `addr_to_integer`. Convierte un vector `addr_word` a entero mediante `to_integer(unsigned)` permitiendo indexar directamente los arreglos ROM y RAM sin conversiones explícitas en cada módulo.

Procedimiento `clear_control_signals`. Desactiva simultáneamente `rom_re`, `ram_we` y `ram_re`. Aunque está declarado en el paquete, el controlador FSM gestiona estas señales directamente en su proceso combinacional con valores por defecto.

3.2 ROM síncrona — rom_sync.vhd

3.2.1. Interfaz

Cuadro 2: Puertos de rom_sync

Puerto	Dirección	Tipo	Descripción
clk	IN	std_logic	Reloj del sistema
re	IN	std_logic	Habilitación de lectura
addr	IN	addr_word	Dirección (4 bits)
data_out	OUT	data_word	Dato leído (8 bits)

3.2.2. Contenido de la ROM

Cuadro 3: Mapa de contenido de la ROM

Dir	Dato	Dir	Dato	Dir	Dato	Dir	Dato
0	AA	4	FF	8	12	12	9A
1	55	5	00	9	34	13	BC
2	F0	6	A5	10	56	14	DE
3	0F	7	5A	11	78	15	EF

3.2.3. Comportamiento

La lectura es síncrona con **un ciclo de latencia**: cuando `re = '1'`, el dato de la posición `addr` se carga en `q_reg` en el siguiente flanco de subida. Esta latencia justifica los dos estados de espera en la FSM (`S_WAIT_ROM` y `S_WAIT_ROM2`). El atributo `romstyle = "M9K"` indica a Quartus que utilice bloques M9K embebidos.

```

1  process(clk)
2  begin
3      if rising_edge(clk) then
4          if re = '1' then
5              q_reg <= rom_mem(addr_to_integer(addr));
6          end if;
7      end if;
8  end process;
9  data_out <= q_reg;
```

Listing 2: Proceso de lectura de la ROM

3.3 RAM síncrona — ram_sincrona.vhd

3.3.1. Interfaz

Cuadro 4: Puertos de ram_sincrona

Puerto	Dirección	Tipo	Descripción
clk	IN	std_logic	Reloj del sistema
rd_en	IN	std_logic	Habilitación de lectura
wr_en	IN	std_logic	Habilitación de escritura
addr	IN	addr_word	Dirección (4 bits)
data_in	IN	data_word	Dato a escribir (8 bits)
data_out	OUT	data_word	Dato leído (8 bits)

3.3.2. Comportamiento

Al igual que la ROM, la RAM tiene **un ciclo de latencia** en lectura, lo que requiere el estado S_READ_RAM_WAIT en la FSM. La escritura también es síncrona y tiene prioridad sobre la lectura.

```

1  process(clk)
2      variable idx : integer range 0 to MEM_DEPTH-1;
3  begin
4      if rising_edge(clk) then
5          idx := addr_to_integer(addr);
6          if wr_en = '1' then
7              mem(idx) <= data_in;          -- Escritura (prioridad alta)
8          elsif rd_en = '1' then
9              q_reg <= mem(idx);           -- Lectura (prioridad baja)
10         end if;
11     end if;
12 end process;

```

Listing 3: Proceso de lectura/escritura de la RAM

3.4 Divisor de frecuencia — divisor_frecuencia.vhd

3.4.1. Interfaz

Cuadro 5: Puertos de divisor_frecuencia

Puerto	Dirección	Tipo	Descripción
clk_50	IN	std_logic	Reloj principal 50 MHz
reset	IN	std_logic	Reset activo en alto
selector	IN	std_logic_vector(1 downto 0)	Selector de velocidad
clk_lento	OUT	std_logic	Reloj dividido de salida

3.4.2. Velocidades disponibles

Cuadro 6: Velocidades del divisor de frecuencia

Selector	Límite de conteo	Frecuencia aproximada
00	24 999 999	≈ 1 Hz
01	12 499 999	≈ 2 Hz
10	6 249 999	≈ 4 Hz
11	3 124 999	≈ 8 Hz

El contador de 26 bits se reinicia al alcanzar el límite e invierte `clk_reg`. La frecuencia resultante es:

$$f_{\text{lento}} = \frac{f_{\text{clk}}}{2 \times (\text{límite} + 1)}$$

3.5 Decodificador BCD a 7 segmentos — `dec_7seg.vhd`

3.5.1. Interfaz

Cuadro 7: Puertos de `dec_7seg`

Puerto	Dirección	Tipo	Descripción
bcd	IN	INTEGER 0..15	Dígito de entrada
seg	OUT	STD_LOGIC_VECTOR(6..0)	Patrón 7 segmentos

3.5.2. Codificación de segmentos

Los displays de la DE0 son **activos en bajo**. El orden de bits es "gfedcba" (bit 6 = segmento *g*, bit 0 = segmento *a*). El módulo soporta los dígitos hexadecimales completos (0–F).

Cuadro 8: Tabla de verdad del decodificador BCD/Hex a 7 segmentos

BCD	Dígito	seg (gfedcba)	BCD	Dígito	seg (gfedcba)
0	0	1000000	8	8	0000000
1	1	1111001	9	9	0010000
2	2	0100100	10	A	0001000
3	3	0110000	11	B	0000011
4	4	0011001	12	C	1000110
5	5	0010010	13	D	0100001
6	6	0000010	14	E	0000110
7	7	1111000	15	F	0001110

3.6 Controlador FSM — `memory_controller.vhd`

3.6.1. Interfaz

Cuadro 9: Puertos de `memory_controller`

Puerto	Dir.	Tipo	Descripción
<code>clk</code>	IN	<code>std_logic</code>	Reloj lento del sistema
<code>rst</code>	IN	<code>std_logic</code>	Reset activo en alto
<code>rom_re</code>	OUT	<code>std_logic</code>	Habilitación lectura ROM
<code>rom_addr</code>	OUT	<code>addr_word</code>	Dirección enviada a ROM
<code>rom_data</code>	IN	<code>data_word</code>	Dato leído desde ROM
<code>ram_we</code>	OUT	<code>std_logic</code>	Habilitación escritura RAM
<code>ram_re</code>	OUT	<code>std_logic</code>	Habilitación lectura RAM
<code>ram_addr</code>	OUT	<code>addr_word</code>	Dirección enviada a RAM
<code>ram_data_in</code>	OUT	<code>data_word</code>	Dato a escribir en RAM
<code>ram_data_out</code>	IN	<code>data_word</code>	Dato leído desde RAM
<code>display_data</code>	OUT	<code>data_word</code>	Dato para visualización (HEX0/HEX1)

3.6.2. Arquitectura interna

El controlador usa **dos procesos separados**, patrón clásico para FSM tipo Moore:

- **Proceso secuencial:** gestiona las transiciones de estado, el contador de dirección (`addr_counter`, 0–15), el registro de dato ROM (`rom_data_reg`) y el registro de display (`display_reg`).
- **Proceso combinacional:** genera las señales de control `s_rom_re`, `s_ram_we` y `s_ram_re` únicamente en función del estado actual (salidas Moore).

Tanto `rom_addr` como `ram_addr` se conectan al mismo `addr_counter`, por lo que ROM y RAM siempre operan sobre la misma dirección.

3.7 Top-level estructural — `memory_system_top.vhd`

3.7.1. Interfaz con la FPGA DE0

Cuadro 10: Puertos del top-level hacia la FPGA DE0

Puerto	Dir.	Tipo	Descripción
<code>CLOCK_50</code>	IN	<code>std_logic</code>	Reloj principal 50 MHz
<code>KEY(0)</code>	IN	<code>std_logic</code>	KEY0: reset activo en bajo
<code>SW(1..0)</code>	IN	<code>std_logic_vector</code>	Selector de velocidad
<code>HEX0</code>	OUT	<code>std_logic_vector(6..0)</code>	Display nibble bajo
<code>HEX1</code>	OUT	<code>std_logic_vector(6..0)</code>	Display nibble alto

3.7.2. Lógica relevante del top-level

Dos aspectos importantes no documentados en los módulos individuales ocurren en el top-level:

Inversión del reset. La KEY0 de la DE0 es activa en bajo. El top-level la invierte para producir la señal `rst` activa en alto que requiere el resto del sistema:

```
1  rst <= not KEY(0);
```

Listing 4: Inversión del reset en el top-level

Separación del byte en nibbles. El dato de 8 bits proveniente del controlador se divide en dos nibbles de 4 bits para alimentar los dos displays independientes:

```
1  hex_low  <= to_integer(unsigned(display_s(3 downto 0)));  -- nibble
    bajo
2  hex_high <= to_integer(unsigned(display_s(7 downto 4)));  -- nibble
    alto
```

Listing 5: Separación de nibbles para HEX0 y HEX1

Por ejemplo, para el primer dato de la ROM (0xAA), HEX1 muestra A (nibble alto) y HEX0 muestra A (nibble bajo).

4 Máquina de estados finita (FSM)

4.1 Descripción general

La FSM implementada en `memory_controller.vhd` es de tipo **Moore**: las salidas dependen únicamente del estado actual. Contiene **9 estados** que cubren las latencias síncronas de ROM y RAM.

4.2 Diagrama de estados

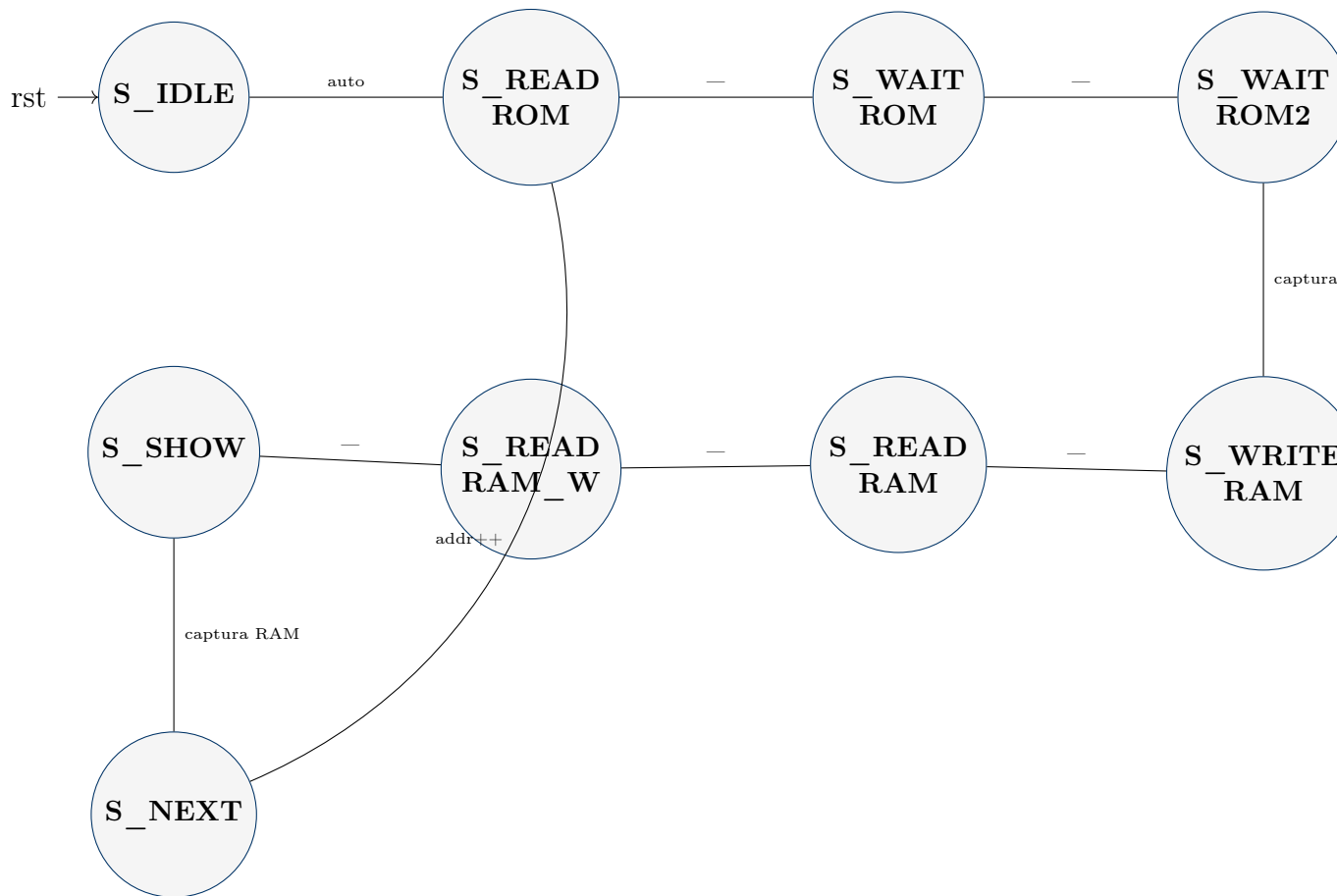


Figura 2: Diagrama de estados completo de la FSM (9 estados).

4.3 Descripción de los 9 estados

S_IDLE	Estado inicial al que se llega por reset. La FSM pasa automáticamente a S_READ_ROM en el siguiente ciclo, sin esperar ninguna señal externa. Durante operación normal no se regresa a este estado .
S_READ_ROM	Se activa <code>rom_re = '1'</code> y se presenta <code>addr_counter</code> a la ROM. La ROM registra la dirección en este flanco pero el dato aún no es válido.
S_WAIT_ROM	<code>rom_re</code> permanece en '1' (primer ciclo de espera). La ROM continúa procesando la dirección. El dato todavía no es válido en la salida.
S_WAIT_ROM2	El dato de la ROM ya es válido. Se captura en <code>rom_data_reg</code> (<code>rom_data_reg <= rom_data</code>). <code>rom_re</code> vuelve a '0'.
S_WRITE_RAM	Se activa <code>ram_we = '1'</code> . El dato capturado en <code>rom_data_reg</code> se escribe en la RAM en la dirección <code>addr_counter</code> .

S_READ_RAM	Se activa <code>ram_re = '1'</code> para iniciar la lectura de verificación. La RAM registra la dirección pero el dato aún no es válido.
S_READ_RAM_WAIT	<code>ram_re</code> permanece en <code>'1'</code> (ciclo de espera de RAM). El dato leído de la RAM estará disponible en el siguiente estado.
S_SHOW	El dato de la RAM ya es válido. Se captura en <code>display_reg</code> (<code>display_reg <= ram_data_out</code>) para su envío al top-level y posterior visualización en los displays HEX0/HEX1.
S_NEXT	Avanza el contador de dirección: si <code>addr_counter = 15</code> lo reinicia a 0, si no lo incrementa en 1. Después regresa directamente a S_READ_ROM para iniciar el siguiente ciclo. El ciclo es continuo e infinito.

4.4 Señales de control por estado

La tabla siguiente refleja exactamente el proceso combinacional del archivo `memory_controller.v`. Nótese que `rom_re` se mantiene activo durante **dos ciclos consecutivos** (S_READ_ROM y S_WAIT_ROM) y `ram_re` igualmente durante dos ciclos (S_READ_RAM y S_READ_RAM_WAIT), cubriendo así la latencia de cada memoria síncrona.

Cuadro 11: Señales de control activas en cada estado (proceso combinacional Moore)

Estado	rom_re	ram_we	ram_re
S_IDLE	0	0	0
S_READ_ROM	1	0	0
S_WAIT_ROM	1	0	0
S_WAIT_ROM2	0	0	0
S_WRITE_RAM	0	1	0
S_READ_RAM	0	0	1
S_READ_RAM_WAIT	0	0	1
S_SHOW	0	0	0
S_NEXT	0	0	0

5 Diagrama de flujo del sistema

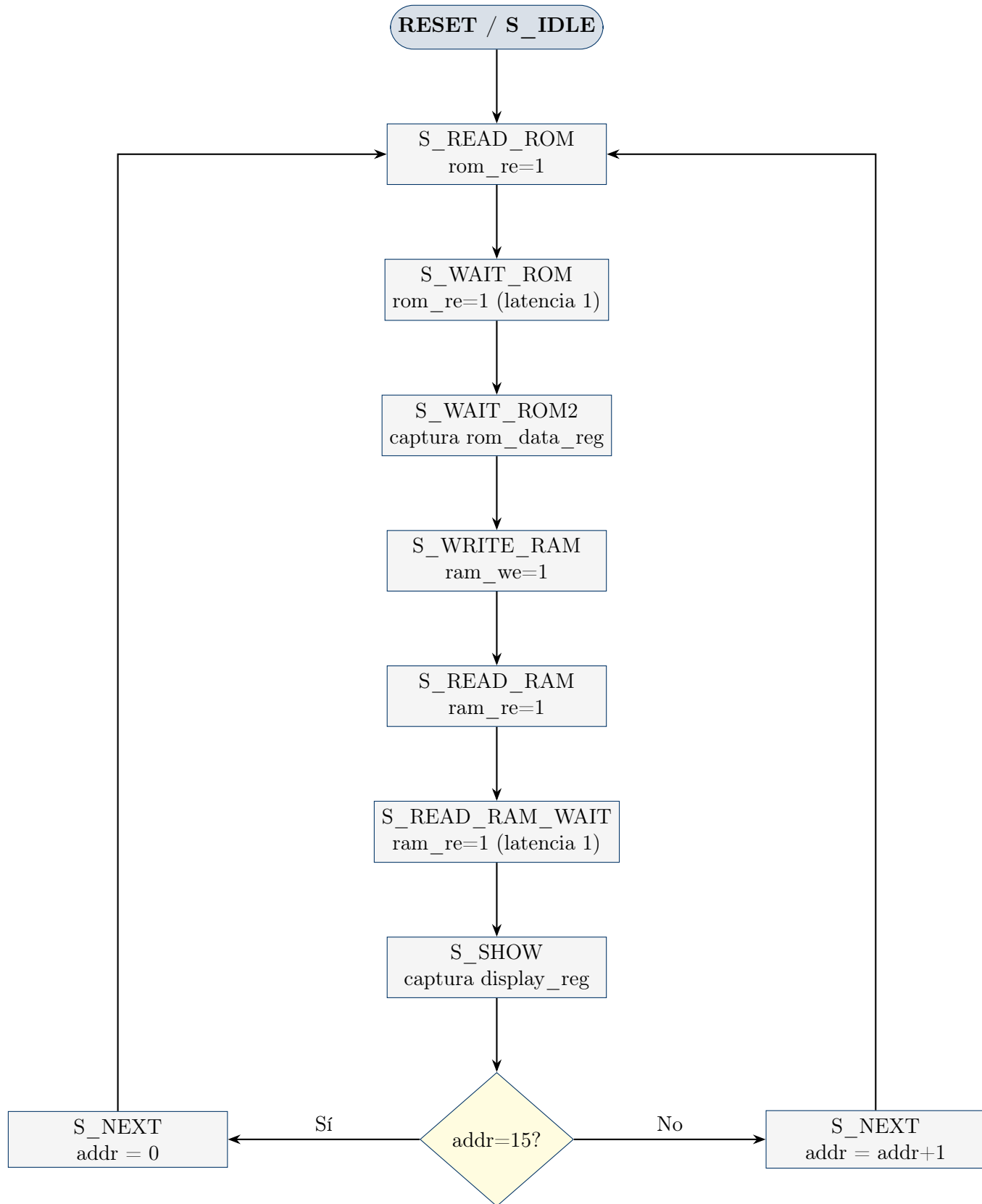


Figura 3: Diagrama de flujo completo del ciclo ROM → RAM → Display (9 estados).

6 Análisis de implementación en FPGA DE0

6.1 Recursos de hardware estimados

Cuadro 12: Recursos estimados para la implementación en Cyclone III

Recurso	Estimado	Descripción
Bloques M9K	2	ROM (1) + RAM (1)
Registros (FF)	≈ 80	FSM (9 estados) + contador + registros de datos + divisor
LUT / LE	≈ 90	Lógica combinacional + decodificadores
Pines de E/S	16	CLOCK_50, KEY(0), SW(1..0), HEX0(6..0), HEX1(6..0)

6.2 Consideraciones de síntesis

- Los atributos `ramstyle = "M9K"` y `romstyle = "M9K"` dirigen a Quartus a implementar las memorias en bloques M9K embebidos, liberando LEs para lógica combinacional.
- El divisor de frecuencia genera `clk_lento` entre 1 Hz y 8 Hz. La FSM opera en este dominio de reloj lento, lo que hace observable cada estado en los displays sin instrumentación adicional.
- La FSM tipo Moore con proceso combinacional separado facilita la inferencia de registros por la herramienta de síntesis.
- El `addr_counter` comparte la misma dirección para ROM y RAM, simplificando el enrutamiento de señales.

7 Simulación y verificación

El archivo `tb_memory_system.vhd` instancia el top-level completo (`memory_system_top`) y ejecuta cuatro pruebas con `SW = "11"` (velocidad máxima, 8 Hz simulados) para reducir el tiempo de simulación.

7.1 Captura de señales en ModelSim

La figura 4 muestra la vista *Wave* de ModelSim durante la ejecución del testbench. Se observan claramente:

- Las dos señales de reloj: `CLOCK_50` (50 MHz) en la parte superior y `clk_lento` (generado por el divisor) en la parte inferior.
- El bus `display_data` (8 bits) que cambia de valor cada vez que la FSM completa un ciclo de lectura/escritura.
- Los patrones `HEX0` y `HEX1` (7 segmentos, activos en bajo) que reflejan los nibbles del dato procesado.
- Las señales de control `rom_re`, `ram_we` y `ram_re` que se activan según la secuencia de estados de la FSM.

- El bus de dirección `addr_counter` que avanza de forma incremental durante el ciclo.

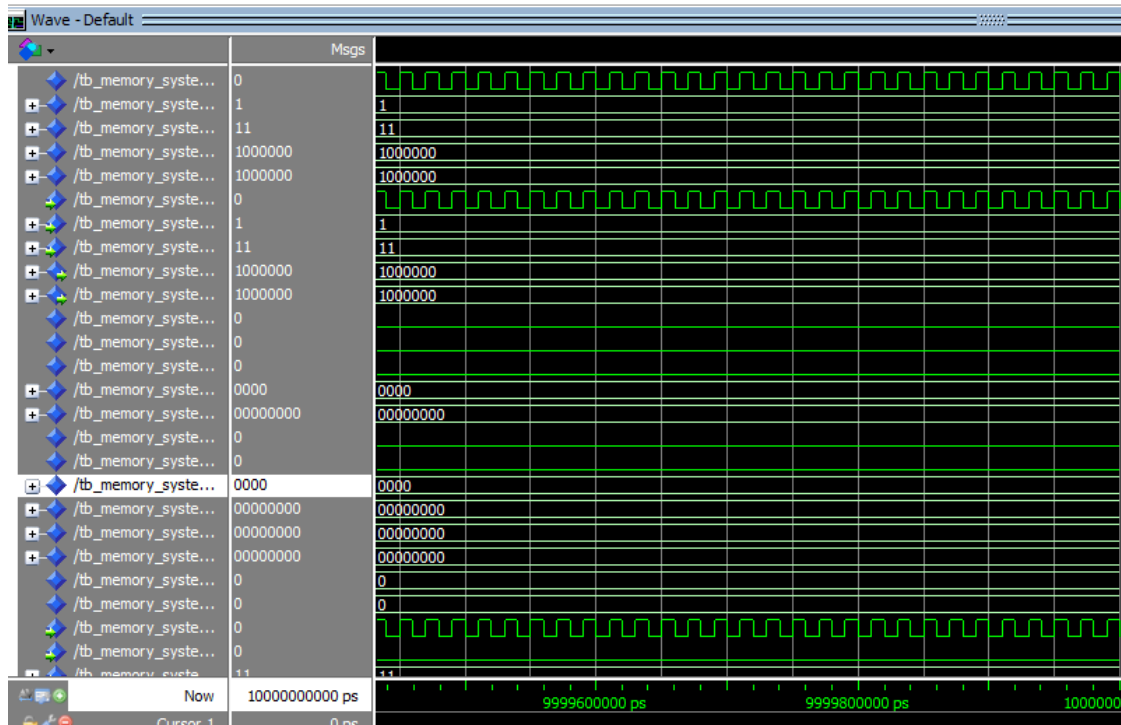


Figura 4: Vista *Wave* de ModelSim: señales del sistema de memoria durante la simulación. Se aprecian los relojes, buses de datos, señales de control de memorias y salidas a displays de 7 segmentos.

7.2 Prueba A — Copia automática ROM → RAM

Se libera el reset y se esperan 3000 ciclos de `CLOCK_50` para que la FSM complete al menos un ciclo completo de los 16 datos. La prueba verifica que el sistema no queda bloqueado y el ciclo ROM→RAM→Display se ejecuta correctamente.

7.3 Prueba B — Verificación del dato en display

Se comprueba que los displays `HEX0` y `HEX1` no estén apagados (`HEX` $\neq$ "1111111") y que muestren exactamente el patrón esperado para el primer dato de la ROM (0xAA): ambos nibbles son A (valor 10), cuyo patrón 7 segmentos es "0001000".

7.4 Prueba C — Persistencia tras reset

Se aplica un reset en caliente y se vuelven a esperar 3000 ciclos. La prueba verifica que la FSM retoma correctamente el ciclo y muestra el mismo dato. Esto es posible porque el reset reinicia la FSM pero **no borra la RAM**; además, la ROM siempre recarga los mismos datos.

7.5 Prueba D — Múltiples resets consecutivos

Se aplican 3 resets consecutivos con 3 ciclos de separación entre ellos, seguidos de 3000 ciclos de operación. La prueba verifica que el sistema permanece estable y los

displays continúan mostrando datos válidos, confirmando que la FSM no se cuelga ante resets rápidos.

8 Conclusiones

El sistema diseñado demuestra las siguientes buenas prácticas en VHDL:

- **Modularidad:** Cada componente tiene responsabilidades bien delimitadas e interfaces claras, lo que facilita la reutilización y la depuración individual.
- **Manejo correcto de latencias:** Los estados dobles de espera (`S_WAIT_ROM/S_WAIT_ROM2` y `S_READ_RAM/S_READ_RAM_WAIT`) garantizan que los datos leídos de memorias síncronas sean válidos antes de ser procesados.
- **FSM tipo Moore:** La separación en proceso secuencial y proceso combinacional produce código más legible y facilita la inferencia correcta por parte de las herramientas de síntesis.
- **Ciclo continuo y autónomo:** La FSM recorre las 16 posiciones de forma indefinida sin intervención externa, regresando a `S_READ_ROM` (no a `S_IDLE`) al finalizar cada ciclo.
- **Observabilidad en hardware:** La separación del byte en nibbles y la instanciación de dos decodificadores permiten visualizar el valor hexadecimal completo (`HEX1:HEX0`) directamente en la FPGA.
- **Robustez ante reset:** El testbench confirma que el sistema se recupera correctamente de resets simples y múltiples resets consecutivos.

Referencias

1. IEEE Standard VHDL Language Reference Manual, *IEEE Std 1076-2008*.
2. P. Ashenden, *The Designer's Guide to VHDL*, 3rd ed., Morgan Kaufmann, 2008.
3. R. Dueck, *Digital Design with CPLD Applications and VHDL*, 2nd ed., Cengage Learning, 2012.
4. Altera Corporation, *DE0 Development and Education Board User Manual*, 2009.
5. Intel/Altera, *Cyclone III Device Handbook*, vol. 1, 2010.